EERoom

Browser-Based Electrical Engineering Simulation Platform

Milestone 4: Coding and Testing (2nd Code Iteration)

Author	Aiden Vargas
Course Number	CST-452 Senior Project II
Instructor	Professor Thompson
Revision	1.0
Date	October 11, 2026
Live Application	https://www.eeroom.io
Source Repository	https://github.com/AidenVargas03/EERoom

Grand Canyon University | College of Engineering and Technology

Abstract

EERoom is a free, browser-based electrical engineering simulation platform that gives students access to an Ohm's Law and power calculator, a waveform visualizer, and a logic gate sandbox without installing software or paying for a license. This document records the second code iteration, in which the three simulation tools, the project dashboard, read-only project sharing, and production deployment were completed and placed on a live server at www.eeroom.io. It contains the Implementation Plan, the Functional Requirements Mapping (Traceability Matrix), the Module Test Cases, the Unit and Integration Test Plans with the Test Analysis Approval Form, and the Source Code Listing required by the CST-452 Capstone Project Handbook. It also documents the scope adjustments made after the first iteration and the justification for each, as the Milestone 4 special note requires. Sixty-eight automated unit tests cover the logic and waveform engines, and all of them pass.

History and Signoff Sheet

Change Record

Date	Author	Revision Notes
10/11/2026	Aiden Vargas	1.0 Milestone 4 submission: second code iteration, test documentation, and scope adjustments

Overall Instructor Feedback and Comments

Integrated instructor feedback into project documentation	☐ Yes ☐ No
Scope adjustments approved (Section 2)	☐ Mentor ☐ Professor Thompson
Milestone approval	☐ Professor Thompson

Contents

1. Iteration Summary
2. Scope Adjustments (Milestone 4 Special Note)
3. Implementation Plan
4. Functional Requirements Mapping (Traceability Matrix)
5. Module Test Cases
6. Unit and Integration Test Plans
7. Test Analysis Approval Form
8. Source Code Listing
9. Application Demonstration

Appendix A: References

1. Iteration Summary

The first code iteration in CST-451 delivered the project scaffolding, the authentication flow, and the Ohm's Law calculator. This second iteration completed every remaining feature in the refined MVP scope and moved the application from a local development environment to a public production deployment.

1.1 Delivered in This Iteration

- Dashboard that lists a user's saved projects newest first, with Open, Delete (with an inline confirmation), quick-launch links, and an empty state for new accounts.
- Waveform Visualizer with sine, square, sawtooth, and triangle signals; amplitude, frequency, phase, and DC offset controls; an oscilloscope-style canvas with labeled axes; RMS and voltage range readouts; and a user-controlled sample rate that demonstrates aliasing below the Nyquist limit.
- Logic Gate Sandbox with seven gate types (AND, OR, NOT, NAND, NOR, XOR, XNOR), click-to-place grid placement, wiring by click or drag, wire removal, input toggling, a live truth table, a derived boolean expression for each output, warnings for unconnected pins, and refusal of feedback loops.
- Save and reopen for all three tools, using one projects table that stores each tool's state as JSONB.
- Read-only sharing: a 64-character share token, a Share button that copies and displays the link, and a public view that renders all three tool types without a login.
- Production deployment: React frontend on Vercel at the custom domain www.eeroom.io, Node.js and Express API on Railway, and PostgreSQL with authentication on Supabase.
- Security hardening: rate limiting of ten requests per fifteen minutes on the public authentication routes, CORS restricted to the production origins, ownership checks on every project query, and secrets excluded from version control.
- Sixty-eight automated unit tests written with Vitest for the logic and waveform engines.

1.2 Technology Stack

Layer	Technology	Hosting
Frontend	React 18, React Router 6, Vite 5, Tailwind CSS 3, Axios	Vercel (www.eeroom.io)
Backend API	Node.js, Express 4, jsonwebtoken, express-rate-limit	Railway
Database and Auth	PostgreSQL and Supabase Auth (bcrypt hashing)	Supabase
Testing	Vitest 1.6 (unit), manual system test scripts	Local and production
Version Control	Git and GitHub (public repository)	github.com/AidenVargas03/EERoom

2. Scope Adjustments (Milestone 4 Special Note)

The handbook allows the scope to be adjusted at Milestone 4 based on lessons learned in CST-451. Every item that was removed or changed is listed below with its justification. Each requires approval from the mentor and the instructor on the History and Signoff Sheet. Requirement IDs refer to the Milestone 2 Requirements Specification.

ID	Change	Requirements Affected	Justification	Status
SC-01	Removed the DC Circuit Simulator (nodal analysis with Kirchhoff's Current Law)	FR-06 to FR-11, US-03	Solving a conductance matrix for arbitrary circuits was the highest-risk feature in the plan and could not be finished and tested by one developer in the remaining timeline. The three remaining tools still cover calculation, signals, and digital logic.	Removed. Pending approval
SC-02	Removed Google OAuth login	FR-02, US-02	Email and password authentication through Supabase Auth meets the authentication requirement. OAuth added a third-party integration with no additional value for an academic MVP.	Removed. Pending approval
SC-03	Removed Stripe payments, the Pro tier, the free-tier save limit, and PDF export	FR-32 to FR-36, US-09 to US-12, PDF Export report	Monetization is not required to demonstrate the platform. All registered users now receive unlimited saves at no cost, which also removes a payment security surface.	Removed. Pending approval
SC-04	Replaced free drag-and-drop placement (react-konva) with click-to-place grid placement drawn in SVG	FR-17, FR-18	Grid cells give every wire a predictable endpoint and make the tool easier to operate during a recorded demonstration. Wires can still be drawn by dragging between pins. konva, react-konva, and mathjs were removed as unused dependencies.	Changed. Pending approval
SC-05	Replaced the three-signal overlay with an ideal-versus-sampled overlay	FR-25	Overlaying the true signal with what a chosen sample rate records teaches aliasing, a core signals concept. One signal per project keeps saved data simple.	Changed. Pending approval
SC-06	Simulation tools require a login	Milestone 3 landing page note	The tools sit behind the protected routes so that every session can save its work. Letting guests try the tools before registering is planned for Milestone 5.	Deferred to Milestone 5
SC-07	Password reset has a backend endpoint but no form yet	FR-04	POST /auth/reset-password is implemented and rate limited. The Forgot Password form on the login page is scheduled for Milestone 5.	Deferred to Milestone 5

2.1 Additions Beyond the Original Plan

- XNOR gate and a derived boolean expression for every logic output.
- Live aliasing demonstration with a Nyquist warning in the waveform tool.

- Rate limiting on the authentication routes and a custom domain.
- A consistent blueprint visual theme across every page, canvas, and drawing.

3. Implementation Plan

Delivery followed a Kanban approach. Backlog items are numbered BL-01 to BL-19 so they are not confused with the user stories numbered US-01 to US-12 in the Requirements Specification. Items moved from Backlog to In Progress to Done, with small commits to GitHub as each task was completed. Estimates were set at the start of the iteration. Actual hours are recorded by the developer from personal time logs and the commit history.

Use Case or User Story	List of Detailed Development Tasks	Estimate (hrs.)	Actual (hrs.)	Percent Complete
BL-01 As a student, I want to register with my email and password so that my work can be saved.	POST /auth/register; mirror the Supabase Auth user into the users table; roll back the auth user if the mirror insert fails; RegisterPage form and validation	4	4	100%
BL-02 As a student, I want to log in and log out securely.	POST /auth/login issuing a 24-hour JWT; separate Supabase sign-in client; useAuth context; ProtectedRoute; LoginPage; logout control	5	7	100%
BL-03 As a student, I want to enter any two of V, I, and R and see the third value and the power instantly.	Solve-for logic based on the two most recently edited fields; zero-division guards; power calculation; unit labels; save and reopen	6	6	100%
BL-04 As a student, I want to visualize a waveform so that I can see how its parameters change it.	utils/waveform.js signal math; four shapes; amplitude, frequency, phase, and offset controls; WaveformCanvas with grid and labeled axes; RMS and range readouts	8	9	100%
BL-05 As a student, I want to choose a sample rate so that I can see aliasing happen.	isAliased Nyquist check; sampled trace overlaid on the ideal curve; on-screen aliasing warning	3	3	100%
BL-06 As a student, I want to build a logic circuit from gates.	Seven-gate palette; click-to-place grid; wiring by click or drag; wire selection and removal; input toggling; Delete key removal	12	15	100%
BL-07 As a student, I want a truth table and expression generated for my circuit.	utils/logicEvaluator.js; generateTruthTable; toExpression; floating-pin reporting; feedback-loop detection; ten-input limit	6	7	100%
BL-08 As a student, I want to save my work in any tool and reopen it later.	api/projects.js wrappers; create versus update using ?project=id; ownership filter on every query	5	5	100%
BL-09 As a student, I want a dashboard of my saved projects.	Newest-first project list; Open and Delete; inline delete confirmation; empty state; quick-launch links	4	4	100%
BL-10 As a student, I want to share a project with a link.	POST /projects/:id/share with a 64-character token; ShareButton that copies and displays the URL	3	3	100%

Use Case or User Story	List of Detailed Development Tasks	Estimate (hrs.)	Actual (hrs.)	Percent Complete
BL-11 As a visitor, I want to open a shared link without an account.	Public GET /share/:token returning no account data; SharedProjectPage for all three tools reusing WaveformCanvas and logicEvaluator; Vercel SPA rewrite	4	5	100%
BL-12 (NFR) As the developer, I want the app deployed and secured.	Vercel, Railway, and Supabase deployment; custom domain; multi-origin CORS; auth rate limiting; .gitignore for secrets	6	8	100%
BL-13 (NFR) As the developer, I want automated unit tests for the core logic.	Vitest setup; 35 logic evaluator tests; 33 waveform tests	5	5	100%
BL-14 As a student, I want to reset a forgotten password.	Backend endpoint complete and rate limited; Forgot Password form not yet built	2	1	50%
BL-15 As the developer, I want Milestone 4 documentation, a manual test run, and a screencast.	This document; execution of manual test cases MT-01 to MT-17; three-part screencast	8	9	100%
		Total estimate	**Total actual**	
		81	91	

Percent of user stories complete for this iteration	14 of 15 user stories complete (93%)
Percent of user stories complete for entire project	14 of 19 planned user stories complete (74%). Remaining for Milestones 5 and 6: BL-14 password reset form, BL-16 server integration tests, BL-17 guest access to the tools, BL-18 responsive and accessibility review, BL-19 final presentation and release.

4. Functional Requirements Mapping (Traceability Matrix)

Each Milestone 2 requirement is mapped to the Milestone 3 Architecture Plan section where it was designed, the class or file and method that implements it, and the test case that verifies it. Test case IDs beginning with UT are automated unit tests. IDs beginning with MT are manual system test cases. Both are defined in Section 5.

Requirement (Use Case or User Story)	Architecture Plan (Milestone 3 Section)	Code Module (Class and Method)	Test Case	Status
FR-01 Register with email and password	4.6 API, 4.7.1 Registration Sequence	authController.register() RegisterPage.jsx, useAuth.register()	MT-01, MT-02	Implemented
FR-02 Log in with Google OAuth	N/A	N/A	N/A	Removed (SC-02)
FR-03 Log out from any page	4.9.2 Dashboard	useAuth.logout() DashboardPage.jsx	MT-05	Implemented
FR-04 Reset password by email	4.6 API	authController.resetPassword()	Planned for Milestone 5	Partial (SC-07)
FR-05 Error message for invalid login	4.6 API	authController.login() LoginPage.jsx	MT-03	Implemented
FR-06 to FR-11 Circuit simulator	N/A	N/A	N/A	Removed (SC-01)
FR-12, FR-13 Enter two of V, I, R and calculate the third	4.8.1 Ohm's Law Algorithm, 4.9.3	OhmToolPage.jsx (solve-for logic)	MT-06	Implemented
FR-14 Calculate power	4.8.1	OhmToolPage.jsx (P = V x I)	MT-06	Implemented
FR-15, FR-16 Instant updates, units, and invalid input handling	4.9.3	OhmToolPage.jsx (zero-division guards)	MT-06, MT-07	Implemented
FR-17 Logic gate components	4.3.3 LogicGateSandbox, 4.9.4	logicEvaluator.GATES, inputCountFor() LogicToolPage.jsx palette	UT-01, UT-02	Implemented (SC-04)
FR-18 Connect gates with wires	4.7.3 Logic Sequence	LogicToolPage.jsx (pin wiring, cablePath()) logicEvaluator.validateCircuit()	UT-06, MT-16	Implemented (SC-04)
FR-19 Evaluate gate outputs	4.8.2 Boolean Evaluation	logicEvaluator.evaluateCircuit()	UT-01, UT-03, UT-04	Implemented
FR-20 Generate the truth table	4.8.2	logicEvaluator.generateTruthTable(), toExpression()	UT-05, UT-07, UT-08	Implemented
FR-21 Toggle inputs between 0 and 1	4.9.4	LogicToolPage.jsx (INPUT click handler)	MT-16	Implemented

Requirement (Use Case or User Story)	Architecture Plan (Milestone 3 Section)	Code Module (Class and Method)	Test Case	Status
FR-22 Select a signal type	4.3.3 WaveformVisualizer	waveform.WAVEFORM_TYPES, sampleAt()	UT-09	Implemented
FR-23 Set amplitude, frequency, and phase	4.3.3	waveform.sampleAt() WaveToolPage.jsx controls	UT-10, UT-11, UT-12	Implemented
FR-24 Render an oscilloscope-style display	4.9.5	WaveformCanvas.drawWaveform() waveform.generateSamples()	UT-13, UT-14, MT-15	Implemented
FR-25 Overlay signals for comparison	4.9.5	waveform.isAliased() WaveformCanvas.jsx (ideal and sampled traces)	UT-15, UT-16, MT-15	Changed (SC-05)
FR-26 Label axes with time and voltage	4.9.5	WaveformCanvas.formatTime(), drawWaveform()	MT-15	Implemented
FR-27 Save a project	4.6 API, 4.7.2 Save Sequence	projectController.createProject(), updateProject() api/projects.js	MT-08	Implemented
FR-28 Load projects from the dashboard	4.6, 4.9.2	projectController.getAllProjects(), getProjectById() DashboardPage.jsx	MT-09	Implemented
FR-29 Delete a project	4.6, 4.9.2	projectController.deleteProject() DashboardPage.jsx	MT-10	Implemented
FR-30 Generate a shareable URL	4.6 API	projectController.generateShareLink() ShareButton.jsx	MT-11	Implemented
FR-31 View a shared project without logging in	4.6, 4.10 Security	shareRoutes.js SharedProjectPage.jsx	MT-12, MT-13	Implemented
FR-32 to FR-36 Free-tier limit and subscriptions	N/A	N/A	N/A	Removed (SC-03)
NFR-04 to NFR-06 Password hashing, HTTPS, 24-hour tokens	4.10 Security Design	Supabase Auth (bcrypt) authMiddleware.requireAuth() authController.login()	MT-04	Implemented
Security Limit login attempts	4.10 Security Design	authRoutes.js (authLimiter)	MT-14	Implemented
Security Users can only access their own projects	4.10 Security Design	projectController.js (.eq on user_id)	MT-17	Implemented

5. Module Test Cases

Test cases follow the handbook template. Unit test cases UT-01 to UT-16 are automated with Vitest. Each step is one automated test, and the results come from the run on October 11, 2026, in which all 68 tests passed. Manual system test cases MT-01 to MT-17 were executed by the developer against the production site on October 11, 2026, and all of them passed.

UT-01: Gate definitions match textbook truth tables

Priority	High
Module	client/src/utils/logicEvaluator.js (GATES)
Test Objective	Validate that each of the seven gate types produces its standard truth table. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	AND produces 0001	Run npx vitest run in the client folder	Every input combination for the gate	Output bits match the textbook table	Pass
2	OR produces 0111	Run npx vitest run in the client folder	Every input combination for the gate	Output bits match the textbook table	Pass
3	NAND produces 1110	Run npx vitest run in the client folder	Every input combination for the gate	Output bits match the textbook table	Pass
4	NOR produces 1000	Run npx vitest run in the client folder	Every input combination for the gate	Output bits match the textbook table	Pass
5	XOR produces 0110	Run npx vitest run in the client folder	Every input combination for the gate	Output bits match the textbook table	Pass
6	XNOR produces 1001	Run npx vitest run in the client folder	Every input combination for the gate	Output bits match the textbook table	Pass
7	NOT produces 10	Run npx vitest run in the client folder	Every input combination for the gate	Output bits match the textbook table	Pass

8	Offers exactly seven gate types	Run npx vitest run in the client folder	Every input combination for the gate	Output bits match the textbook table	Pass

UT-02: Pin counts per component

Priority	Medium
Module	logicEvaluator.js (inputCountFor)
Test Objective	Validate that each component exposes the correct number of input pins. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Gives NOT one pin and the binary gates two	Run npx vitest run in the client folder	Each component type	NOT has 1 pin, binary gates 2, INPUT 0, OUTPUT 1	Pass
2	Gives INPUT no pins and OUTPUT one	Run npx vitest run in the client folder	Each component type	NOT has 1 pin, binary gates 2, INPUT 0, OUTPUT 1	Pass

UT-03: Unconnected pins read as 0 and are reported

Priority	High
Module	logicEvaluator.js (evaluateCircuit)
Test Objective	Validate the error-condition behavior of a gate with a floating input. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Gives AND a 0 output when its second pin is floating and A is high	Run npx vitest run in the client folder	Circuit with one unwired pin	Floating pin evaluates as 0 and is named in the report	Pass
2	Gives OR a 1 output in the same situation	Run npx vitest run in the client folder	Circuit with one unwired pin	Floating pin evaluates as 0 and is named in the report	Pass
3	Names the floating pin so the interface can flag it	Run npx vitest run in the client folder	Circuit with one unwired pin	Floating pin evaluates as 0 and is named in the report	Pass
4	Reports nothing floating once every pin is wired	Run npx vitest run in the client folder	Circuit with one unwired pin	Floating pin evaluates as 0 and is named in the report	Pass

UT-04: Feedback loops are refused

Priority	High
Module	logicEvaluator.js (evaluateCircuit, generateTruthTable)
Test Objective	Validate that a circuit with a loop raises an error instead of returning a guessed value. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Throws when evaluating a loop	Run npx vitest run in the client folder	Circuit with a cycle	An error is thrown	Pass
2	Throws when building a truth table from a loop	Run npx vitest run in the client folder	Circuit with a cycle	An error is thrown	Pass
3	Throws on a gate wired directly back to itself	Run npx vitest run in the client folder	Circuit with a cycle	An error is thrown	Pass

UT-05: Truth table size limits

Priority	Medium
Module	logicEvaluator.js (generateTruthTable)
Test Objective	Validate the ten-input limit and the row ordering of the truth table. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Allows 10 inputs	Run npx vitest run in the client folder	Circuits with 0, 10, and 11 inputs	Within the limit a table is built; above it an error is raised	Pass
2	Refuses one more than the limit	Run npx vitest run in the client folder	Circuits with 0, 10, and 11 inputs	Within the limit a table is built; above it an error is raised	Pass
3	Still gives one row for a circuit with no inputs	Run npx vitest run in the client folder	Circuits with 0, 10, and 11 inputs	Within the limit a table is built; above it an error is raised	Pass
4	Counts rows up in binary with the first input as the most significant bit	Run npx vitest run in the client folder	Circuits with 0, 10, and 11 inputs	Within the limit a table is built; above it an error is raised	Pass

UT-06: Structural validation errors

Priority	High
Module	logicEvaluator.js (validateCircuit)

Test Objective	Validate that malformed circuits are detected and reported. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Catches a duplicate node id	Run npx vitest run in the client folder	Malformed circuit objects	Each structural problem is reported; a valid circuit reports none	Pass
2	Catches an unknown gate type	Run npx vitest run in the client folder	Malformed circuit objects	Each structural problem is reported; a valid circuit reports none	Pass
3	Catches a wire landing on a pin the gate does not have	Run npx vitest run in the client folder	Malformed circuit objects	Each structural problem is reported; a valid circuit reports none	Pass
4	Catches two wires driving one pin, which is a short rather than a circuit	Run npx vitest run in the client folder	Malformed circuit objects	Each structural problem is reported; a valid circuit reports none	Pass
5	Catches a wire into an INPUT component	Run npx vitest run in the client folder	Malformed circuit objects	Each structural problem is reported; a valid circuit reports none	Pass
6	Reports nothing for a well-formed circuit	Run npx vitest run in the client folder	Malformed circuit objects	Each structural problem is reported; a valid circuit reports none	Pass

UT-07: Column ordering and half adder

Priority	Medium
Module	logicEvaluator.js (getInputNodes, getOutputNodes)
Test Objective	Validate column order and correctness against a half adder computed by hand. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Follows the board, so Sum placed above Carry gives columns S then C	Run npx vitest run in the client folder	Half adder circuit (XOR and AND)	Columns follow board order; results match the hand-computed table	Pass
2	Falls back to label order when nodes have no board position	Run npx vitest run in the client folder	Half adder circuit (XOR and AND)	Columns follow board order; results match the hand-computed table	Pass
3	Separates inputs from outputs	Run npx vitest run in the client folder	Half adder circuit (XOR and AND)	Columns follow board order; results match the hand-computed table	Pass
4	Matches the truth table computed by hand	Run npx vitest run in the client folder	Half adder circuit (XOR and AND)	Columns follow board order; results match the hand-computed table	Pass
5	Derives expressions that agree with the gates used	Run npx vitest run in the client folder	Half adder circuit (XOR and AND)	Columns follow board order; results match the hand-computed table	Pass

6	Carries only when both inputs are high	Run npx vitest run in the client folder	Half adder circuit (XOR and AND)	Columns follow board order; results match the hand-computed table	Pass

UT-08: Boolean expression generation

Priority	Medium
Module	logicEvaluator.js (toExpression)
Test Objective	Validate that the derived expression matches how the circuit is wired and evaluated. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Nests sub-expressions in the order the gates are wired	Run npx vitest run in the client folder	Nested gate circuits	Expression nests in wiring order and writes a floating pin as 0	Pass
2	Writes an unconnected pin as 0, matching how it is evaluated	Run npx vitest run in the client folder	Nested gate circuits	Expression nests in wiring order and writes a floating pin as 0	Pass

UT-09: Waveform shape definitions

Priority	High
Module	client/src/utils/waveform.js (sampleAt)
Test Objective	Validate the four waveform shapes at key points in the cycle. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Starts every shape in phase at t=0, except square which has no rising zero crossing	Run npx vitest run in the client folder	t at quarter and half cycles	Each shape produces its expected values	Pass
2	Puts a 1 Hz sine through 0, +1, 0, -1 at the quarter points	Run npx vitest run in the client folder	t at quarter and half cycles	Each shape produces its expected values	Pass
3	Peaks the triangle at the quarter points and crosses zero at the half	Run npx vitest run in the client folder	t at quarter and half cycles	Each shape produces its expected values	Pass
4	Flips the square wave at the half cycle	Run npx vitest run in the client folder	t at quarter and half cycles	Each shape produces its expected values	Pass
5	Ramps the sawtooth and resets it at the half cycle	Run npx vitest run in the client folder	t at quarter and half cycles	Each shape produces its expected values	Pass

6	Exposes exactly the four supported shapes	Run npx vitest run in the client folder	t at quarter and half cycles	Each shape produces its expected values	Pass

UT-10: Waveform parameters

Priority	High
Module	waveform.js (sampleAt)
Test Objective	Validate amplitude, offset, and phase handling. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Scales by amplitude	Run npx vitest run in the client folder	Amplitude, offset, phase values	Output scales, shifts, and phase-shifts correctly	Pass
2	Shifts by DC offset	Run npx vitest run in the client folder	Amplitude, offset, phase values	Output scales, shifts, and phase-shifts correctly	Pass
3	Treats 90 degrees of phase as a quarter cycle	Run npx vitest run in the client folder	Amplitude, offset, phase values	Output scales, shifts, and phase-shifts correctly	Pass
4	Wraps negative phase into the cycle rather than leaving it out of range	Run npx vitest run in the client folder	Amplitude, offset, phase values	Output scales, shifts, and phase-shifts correctly	Pass

UT-11: Waveform edge cases

Priority	Medium
Module	waveform.js (sampleAt)
Test Objective	Validate behavior at 0 Hz, zero amplitude, and negative frequency. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Holds a constant at 0 Hz, because the cycle never advances	Run npx vitest run in the client folder	0 Hz, 0 V, negative Hz	Constant, flat, and mirrored outputs	Pass
2	Produces a flat line at zero amplitude	Run npx vitest run in the client folder	0 Hz, 0 V, negative Hz	Constant, flat, and mirrored outputs	Pass
3	Mirrors the wave for a negative frequency	Run npx vitest run in the client folder	0 Hz, 0 V, negative Hz	Constant, flat, and mirrored outputs	Pass

UT-12: Waveform error conditions

Priority	High
Module	waveform.js (sampleAt)
Test Objective	Validate that invalid input is rejected instead of silently coerced. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Rejects an unknown waveform type	Run npx vitest run in the client folder	Unknown type, NaN, Infinity, numeric strings	An error is thrown for each	Pass
2	Rejects a non-finite time	Run npx vitest run in the client folder	Unknown type, NaN, Infinity, numeric strings	An error is thrown for each	Pass
3	Rejects non-finite parameters	Run npx vitest run in the client folder	Unknown type, NaN, Infinity, numeric strings	An error is thrown for each	Pass
4	Rejects a numeric string rather than silently coercing it	Run npx vitest run in the client folder	Unknown type, NaN, Infinity, numeric strings	An error is thrown for each	Pass

UT-13: Sample generation

Priority	High
Module	waveform.js (generateSamples)
Test Objective	Validate sample count, ordering, and handling of invalid durations and rates. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Includes both endpoints, so one second at 1000 Hz is 1001 samples	Run npx vitest run in the client folder	1 s at 1000 Hz; non-positive values	1001 ordered samples; empty for zero duration; error for a bad rate	Pass
2	Returns times in ascending order	Run npx vitest run in the client folder	1 s at 1000 Hz; non-positive values	1001 ordered samples; empty for zero duration; error for a bad rate	Pass
3	Treats a non-positive duration as nothing to draw, not an error	Run npx vitest run in the client folder	1 s at 1000 Hz; non-positive values	1001 ordered samples; empty for zero duration; error for a bad rate	Pass

4	Rejects a non-positive sample rate, which is incoherent rather than empty	Run npx vitest run in the client folder	1 s at 1000 Hz; non-positive values	1001 ordered samples; empty for zero duration; error for a bad rate	Pass

UT-14: RMS and voltage range

Priority	Medium
Module	waveform.js (rms, voltageRange)
Test Objective	Validate the readout calculations. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Gives a sine peak divided by the square root of 2	Run npx vitest run in the client folder	Sine and square samples; empty set	Sine RMS = peak / sqrt(2); square RMS = amplitude; empty set returns 0	Pass
2	Gives a square wave its own amplitude	Run npx vitest run in the client folder	Sine and square samples; empty set	Sine RMS = peak / sqrt(2); square RMS = amplitude; empty set returns 0	Pass
3	Returns 0 for an empty set rather than NaN	Run npx vitest run in the client folder	Sine and square samples; empty set	Sine RMS = peak / sqrt(2); square RMS = amplitude; empty set returns 0	Pass
4	Spans offset plus and minus amplitude	Run npx vitest run in the client folder	Sine and square samples; empty set	Sine RMS = peak / sqrt(2); square RMS = amplitude; empty set returns 0	Pass
5	Returns zeroes for an empty set	Run npx vitest run in the client folder	Sine and square samples; empty set	Sine RMS = peak / sqrt(2); square RMS = amplitude; empty set returns 0	Pass

UT-15: Nyquist limit check

Priority	High
Module	waveform.js (isAliased)
Test Objective	Validate when a sample rate is flagged as aliased. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Passes a signal sampled well above twice its frequency	Run npx vitest run in the client folder	Rates above, at, and below 2f	Flagged at or below 2f, never for DC	Pass
2	Flags a signal sampled below twice its frequency	Run npx vitest run in the client folder	Rates above, at, and below 2f	Flagged at or below 2f, never for DC	Pass

3	Flags a signal sampled at exactly twice its frequency, since the theorem needs strictly more	Run npx vitest run in the client folder	Rates above, at, and below 2f	Flagged at or below 2f, never for DC	Pass
4	Uses the magnitude of a negative frequency	Run npx vitest run in the client folder	Rates above, at, and below 2f	Flagged at or below 2f, never for DC	Pass
5	Never flags DC	Run npx vitest run in the client folder	Rates above, at, and below 2f	Flagged at or below 2f, never for DC	Pass

UT-16: Aliasing produces a ghost frequency

Priority	Medium
Module	waveform.js (generateSamples)
Test Objective	Validate that undersampling produces the expected alias frequency. (Automated, Vitest)

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Shows the true frequency when properly sampled	Run npx vitest run in the client folder	f sampled above and below Nyquist	True frequency when sampled correctly; ghost at |f - fs| when undersampled	Pass
2	Shows a ghost at |f - fs| when undersampled	Run npx vitest run in the client folder	f sampled above and below Nyquist	True frequency when sampled correctly; ghost at |f - fs| when undersampled	Pass

MT-01: Register a new account

Priority	High
Module	RegisterPage.jsx, authController.register()
Test Objective	Validate that a new user can register with a valid email and password.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Open register page	Go to https://www.eeroom.io and click Sign up	N/A	Registration form is shown	Pass 10/11/26
2	Submit valid details	Enter name, email, and password, then submit	New email, password of 8+ characters	Redirected to the login page with no error	Pass 10/11/26

3	Confirm stored user	Open the users table in Supabase	Same email	One new row exists for the email	Pass 10/11/26

MT-02: Reject a duplicate registration

Priority	High
Module	authController.register()
Test Objective	Validate that registration fails when the email is already registered.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Register a used email	Submit the registration form again with the MT-01 email	Existing email	An error message is shown and no second account is created	Pass 10/11/26

MT-03: Reject an invalid login

Priority	High
Module	LoginPage.jsx, authController.login()
Test Objective	Validate that a wrong password is refused with a clear message.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Wrong password	Enter the registered email with a wrong password	Valid email, wrong password	Message: Invalid email or password. User stays on the login page	Pass 10/11/26
2	Correct password	Enter the correct password	Valid credentials	Dashboard opens	Pass 10/11/26

MT-04: Protect private pages

Priority	High
Module	ProtectedRoute.jsx, authMiddleware.requireAuth()
Test Objective	Validate that private pages and API routes cannot be used without logging in.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail

1	Open dashboard logged out	In a private window, go to https://www.eeroom.io/dashboard	No session	Redirected to the login page	Pass 10/11/26
2	Call API without a token	Send GET /projects to the API with no Authorization header	No token	401 Unauthorized	Pass 10/11/26

MT-05: Log out

Priority	Medium
Module	useAuth.logout(), DashboardPage.jsx
Test Objective	Validate that logging out ends the session.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Log out	Click Log out on the dashboard	Logged-in user	Token is cleared and the login page is shown	Pass 10/11/26
2	Try the back button	Press Back to the dashboard	N/A	Redirected to the login page	Pass 10/11/26

MT-06: Calculate with Ohm's Law

Priority	High
Module	OhmToolPage.jsx
Test Objective	Validate that any two values produce the third value and the power instantly.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Solve for current	Enter V and R	V = 12, R = 4	I = 3 A, P = 36 W	Pass 10/11/26
2	Solve for resistance	Clear R, enter V and I	V = 9, I = 3	R = 3 Ω, P = 27 W	Pass 10/11/26
3	Solve for voltage	Clear V, enter I and R	I = 2, R = 5	V = 10 V, P = 20 W	Pass 10/11/26

MT-07: Handle division by zero

Priority	High
Module	OhmToolPage.jsx
Test Objective	Validate the error condition when a zero would cause division by zero.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Zero current	Enter V and a current of 0	V = 5, I = 0	Message: Current cannot be 0 when solving for resistance	Pass 10/11/26
2	Zero resistance	Enter V and a resistance of 0	V = 5, R = 0	Message: Resistance cannot be 0 when solving for current	Pass 10/11/26

MT-08: Save a new project

Priority	High
Module	projectController.createProject(), api/projects.js
Test Objective	Validate that tool state is saved as a named project.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Save	In the Ohm tool, name the project and click Save	Name: Test Ohm 1	Save confirmation; URL now includes ?project=	Pass 10/11/26
2	Save again	Change a value and click Save	New value	Same project is updated; no duplicate on the dashboard	Pass 10/11/26

MT-09: Reopen a project from the dashboard

Priority	High
Module	DashboardPage.jsx, projectController.getProjectById()
Test Objective	Validate that saved projects are listed and reopen with their saved state.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	View list	Open the dashboard	Projects from MT-08 and MT-15	Projects listed newest first with tool type and date	Pass 10/11/26

2	Open	Click Open on a project	Test Ohm 1	Tool opens with the saved values	Pass 10/11/26

MT-10: Delete a project

Priority	Medium
Module	DashboardPage.jsx, projectController.deleteProject()
Test Objective	Validate that deletion asks for confirmation and removes the project.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Cancel delete	Click Delete, then Cancel	Any project	Confirm and Cancel buttons appear; after Cancel the project remains	Pass 10/11/26
2	Confirm delete	Click Delete, then Confirm	Same project	Project is removed from the list and the database	Pass 10/11/26

MT-11: Create a share link

Priority	High
Module	ShareButton.jsx, projectController.generateShareLink()
Test Objective	Validate that a saved project produces a shareable link.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Share	Open a saved project and click Share	Saved project	A /share/ link is displayed and copied to the clipboard	Pass 10/11/26

MT-12: View a shared project logged out

Priority	High
Module	shareRoutes.js, SharedProjectPage.jsx
Test Objective	Validate read-only access for a visitor with no account.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail

1	Open link	Paste the MT-11 link into a private window	Share URL	Project renders read only with no login prompt	Pass 10/11/26
2	Repeat for each tool	Share and open an Ohm, waveform, and logic project	Three share URLs	Each tool type renders correctly	Pass 10/11/26

MT-13: Reject an invalid share link

Priority	Medium
Module	shareRoutes.js
Test Objective	Validate the error condition for a token that does not exist.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Bad token	Go to https://www.eeroom.io/share/notarealtoken	Invalid token	Message: This link is not valid. No project data is shown	Pass 10/11/26

MT-14: Rate limit login attempts

Priority	High
Module	authRoutes.js (authLimiter)
Test Objective	Validate that repeated login attempts are blocked.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Repeat failed logins	Submit a wrong password 11 times within 15 minutes	Valid email, wrong password	The 11th attempt is refused with a too-many-requests message	Pass 10/11/26

MT-15: Visualize a waveform and its aliasing

Priority	High
Module	WaveToolPage.jsx, WaveformCanvas.jsx
Test Objective	Validate drawing, axis labels, readouts, and the aliasing warning.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail

1	Draw each shape	Select sine, square, sawtooth, and triangle	5 V, 50 Hz	Each shape draws with labeled time and voltage axes	Pass 10/11/26
2	Healthy sample rate	Set the sample rate	1000 Hz	Sampled trace sits on the ideal curve; no warning	Pass 10/11/26
3	Undersample	Lower the sample rate	60 Hz	Sampled trace departs from the curve and an aliasing warning appears	Pass 10/11/26

MT-16: Build and evaluate a logic circuit

Priority	High
Module	LogicToolPage.jsx
Test Objective	Validate placement, wiring, toggling, the truth table, and floating-pin warnings.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Place components	Place two INPUTs, an AND gate, and an OUTPUT	A, B, AND, Y	Components snap to grid cells	Pass 10/11/26
2	Leave a pin unwired	Wire only A into the AND gate	N/A	The open pin is ringed in amber and a warning reports 1 unconnected input pin	Pass 10/11/26
3	Finish wiring	Wire B and the AND output to Y	N/A	Truth table shows Y = 1 only when A = 1 and B = 1; expression is Y = (A AND B)	Pass 10/11/26
4	Toggle inputs	Click A and B to set both to 1	A = 1, B = 1	Output Y shows 1	Pass 10/11/26
5	Remove a wire	Select a wire and delete it	Any wire	Wire is removed and results update	Pass 10/11/26

MT-17: Block access to another user's project

Priority	High
Module	projectController.js (ownership filter)
Test Objective	Validate that a user cannot open a project owned by another account.

Step	Test Name	Test Steps	Test Data	Expected Results	Pass/Fail
1	Open another user's ID	Log in as user B and open /tools/ohm?project=<user A's project id>	Two test accounts	API returns 404 Project not found; user A's data is not shown	Pass 10/11/26

6. Unit and Integration Test Plans

6.1 Test Approach

EERoom is tested in layers. The calculation engines that contain the most logic, utils/logicEvaluator.js and utils/waveform.js, were written as pure functions with no React or database code so that they could be unit tested in isolation. Every function is tested for normal input, edge cases, and error conditions. Components are then tested together through the user interface and the API, and the whole system is tested end to end against the production deployment using the manual test cases in Section 5. Defects found at any stage are returned to development, fixed, and retested.

6.2 Test Tools

#	Tool	Purpose
1	Vitest 1.6	Automated unit tests for the logic and waveform engines
2	Google Chrome and DevTools	Manual system testing, network inspection, and responsive layout checks
3	Supabase dashboard	Verifying rows created, updated, and deleted by the API
4	Browser private window	Testing logged-out access and shared links
5	GitHub	Version history and traceability of fixes

6.3 Component Testing

Component	How It Is Tested	Test Cases
Logic evaluator	Automated unit tests	UT-01 to UT-08 (35 tests)
Waveform engine	Automated unit tests	UT-09 to UT-16 (33 tests)
Authentication API and pages	Manual tests through the UI and direct API calls	MT-01 to MT-05, MT-14
Projects API and dashboard	Manual tests through the UI and the Supabase dashboard	MT-08 to MT-10, MT-17
Sharing	Manual tests in a private window	MT-11 to MT-13
Tool pages	Manual tests of each tool	MT-06, MT-07, MT-15, MT-16

6.4 Integration Testing

Integration tests confirm that the components work together across the frontend, the Express API, and Supabase. They are run as manual end-to-end flows: register and log in (MT-01, MT-03), save a project from a tool and see it appear on the dashboard (MT-08, MT-09), share it and open it logged out (MT-11, MT-12), and confirm that protected routes and ownership rules hold (MT-04, MT-17). Automated integration tests for the API with Jest and Supertest are planned for Milestone 5 (BL-16).

6.5 Requirements Testing

Every functional requirement in scope maps to at least one test case in the Traceability Matrix (Section 4). Requirement tests cover normal conditions and error conditions, for example: validate that a project can be saved (MT-08); validate that saving again updates rather than duplicates (MT-08 step 2); validate that registration fails when the email already exists (MT-02); and validate that the truth table is refused above ten inputs (UT-05).

6.6 System Test Plan

Part 1: Overview

Date	October 11, 2026
Objectives and goals	Confirm that the deployed application meets the in-scope functional requirements, the security design, and the business process of create, save, reopen, and share.
Duration	One test session of about two hours, repeated after any defect fix

Part 2: Scope and Resources

Features to be tested	Registration, login, logout, protected routes, rate limiting, Ohm's Law calculator, Waveform Visualizer, Logic Gate Sandbox, save and update, dashboard, delete, share link creation, public shared view, ownership checks
Features not tested	Removed features (circuit simulator, OAuth, payments) and the deferred password reset form
Resources	Developer (Aiden Vargas), two test accounts, production site at www.eeroom.io, Railway API, Supabase project

Part 3: Strategy

Procedures	Execute MT-01 to MT-17 in order, record Pass or Fail and the date, log any defect in the GitHub repository, fix it, and rerun the affected cases
Environment	Production: Vercel frontend, Railway API, Supabase database, latest Google Chrome on desktop
Tools	Chrome DevTools, Supabase dashboard, private browser window, Vitest
Pass criteria	All 68 unit tests pass and every High priority manual test case passes

Entry Criteria for System Testing

- Unit testing finished: met. 68 of 68 unit tests pass.
- Modules fully integrated: met. All three tools, the dashboard, and sharing run against the live API.
- Development complete per the specification: met for the adjusted scope in Section 2.
- Testing environment available: met. The production deployment is used as the staging environment.

6.7 Acceptance Testing

EERoom is a self-directed project, so there is no client. Acceptance is defined as the instructor and mentor approving the system test results and the scope adjustments through the Test Analysis Approval Form in Section 7. Informal usability feedback from classmates will be gathered during Milestone 5.

7. Test Analysis Approval Form

Acceptance Test Final Report

Project name	EERoom: Browser-Based Electrical Engineering Simulation Platform
Date	October 11, 2026
General description of the test effort	68 automated unit tests across the logic and waveform engines (all passing) and 17 manual system test cases executed against the production deployment.
Unit test results	68 passed, 0 failed (Vitest 1.6, run on 10/11/2026)
Manual system test results	17 of 17 passed, 0 failed (35 of 35 steps). Date run: 10/11/2026

Unresolved Defects and Known Limitations

Issue or Defect	Impact (H, M, L)	Risk Mitigation	Work Around
Password reset form not built (SC-07)	M	Scheduled for Milestone 5	Endpoint exists and works when called directly
No automated tests for the server API	M	Manual cases MT-01 to MT-17 cover every route; Jest and Supertest planned for Milestone 5	Manual testing
Visitors cannot try the tools before registering (SC-06)	L	Guest access planned for Milestone 5	Registration takes under a minute

Final Sign-Off and Approval

Role	Name	Signature	Date
Developer	Aiden Vargas	Aiden Vargas	10/11/2026
Mentor			
Instructor	Professor Thompson		

8. Source Code Listing

The complete source code is published at https://github.com/AidenVargas03/EERoom. This listing reflects the repository as of October 9, 2026. Every source file opens with a header comment describing its purpose. Code adapted from official documentation or other external sources is marked inline with a Source comment and URL, as described in the repository README. Business logic designed by the developer, such as the Ohm's Law solve-for logic, the boolean evaluator, and the project ownership checks, is not cited because it is original.

8.1 Frontend (client)

File	Lines	Description
src/main.jsx	29	Application entry point. Mounts React with the router and the authentication provider.
src/App.jsx	77	Route table for every page; wraps private pages in ProtectedRoute.
src/theme.js	66	Single source of the color palette used by Tailwind, the waveform canvas, and the logic board.
src/api/client.js	35	Shared Axios instance that sets the API base URL and attaches the JWT to every request.
src/api/projects.js	59	Functions for the /projects endpoints: list, get, create, update, delete, share.
src/hooks/useAuth.jsx	77	React context that stores the logged-in user and token and provides register, login, and logout.
src/components/ProtectedRoute.jsx	36	Redirects to the login page when no user is logged in.
src/components/ShareButton.jsx	100	Creates a share link for a saved project, copies it, and displays it.
src/components/WaveformCanvas.jsx	232	Draws the grid, axes, ideal curve, and sampled trace on an HTML5 canvas. Shared by the editor and the shared view.
src/pages/LandingPage.jsx	284	Public front page with a working aliasing illustration and links to sign up.
src/pages/RegisterPage.jsx	99	Registration form.
src/pages/LoginPage.jsx	89	Login form with error messages from the API.
src/pages/DashboardPage.jsx	270	Lists the user's projects with Open and Delete, quick-launch links, and an empty state.
src/pages/OhmToolPage.jsx	375	Ohm's Law and power calculator with save and reopen.
src/pages/WaveToolPage.jsx	353	Waveform Visualizer controls, readouts, aliasing warning, and save and reopen.
src/pages/LogicToolPage.jsx	864	Logic Gate Sandbox: grid placement, wiring, toggling, truth table, expressions, and save and reopen.
src/pages/SharedProjectPage.jsx	348	Public read-only view of a shared project for all three tools.

File	Lines	Description
src/utils/logicEvaluator.js	395	Gate definitions, circuit validation, evaluation, truth table generation, and boolean expressions.
src/utils/waveform.js	220	Signal generation, sampling, RMS, voltage range, and the Nyquist check.
src/utils/logicEvaluator.test.js	381	35 unit tests for the logic evaluator.
src/utils/waveform.test.js	257	33 unit tests for the waveform engine.
vercel.json	9	Rewrites every path to index.html so deep links and share URLs load on Vercel.

8.2 Backend (server)

File	Lines	Description
index.js	99	Express entry point: CORS for the production origins, JSON parsing, proxy trust, and route mounting.
config/supabaseClient.js	72	Two Supabase clients: a service-role client for data and a separate client for user sign-in.
middleware/authMiddleware.js	53	requireAuth verifies the JWT bearer token and attaches the user to the request.
routes/authRoutes.js	56	Public auth routes with rate limiting of 10 requests per 15 minutes.
routes/projectRoutes.js	32	Protected project routes; requireAuth runs on each one.
routes/shareRoutes.js	35	Public GET /share/:token returning only the shared project's content.
controllers/authController.js	152	register, login, and resetPassword.
controllers/projectController.js	179	Project create, read, update, delete, and share, each filtered by the owner's user ID.
db/schema.sql	46	PostgreSQL schema for the users and projects tables.

8.3 Configuration

package.json files define dependencies and scripts for each folder. vite.config.js, tailwind.config.js, and postcss.config.js configure the frontend build. .env.example files list the required environment variables; the real .env files are excluded from the repository by .gitignore.

9. Application Demonstration

A three-part screencast, about 12 minutes in total, demonstrates every feature completed in this iteration on the live site, www.eeroom.io: registration and login, the three simulation tools, saving and reopening from the dashboard, sharing a project, and opening the shared link without an account.

Screencast, Part 1	https://www.loom.com/share/01b71bfcc21c4aacad6fe3a17a616b9c
Screencast, Part 2	https://www.loom.com/share/2041926f14054a9da90aea7e5e2151ab
Screencast, Part 3	https://www.loom.com/share/14c9ac4205c64851b303c64d57261d63
Length	Approximately 12 minutes total

Appendix A: References

Express rate limit. (2026). Troubleshooting proxy issues. GitHub. https://github.com/express-rate-limit/express-rate-limit/wiki/Troubleshooting-Proxy-Issues

Grand Canyon University. (2026). CST-452 capstone project handbook.

React Router. (2026). useSearchParams. https://reactrouter.com/en/main/hooks/use-search-params

Supabase. (2026). JavaScript client library. https://supabase.com/docs/reference/javascript

Vercel. (2026). Rewrites. https://vercel.com/docs/projects/project-configuration

Vitest. (2026). Getting started. https://vitest.dev/guide/